TEACHING SERIES · INTRO TO AI SYSTEMS

Retrieval-Augmented Generation (RAG)

Focus: Embeddings & Retrieval — with worked examples

Part 2 · After chunking — how search actually finds the right text

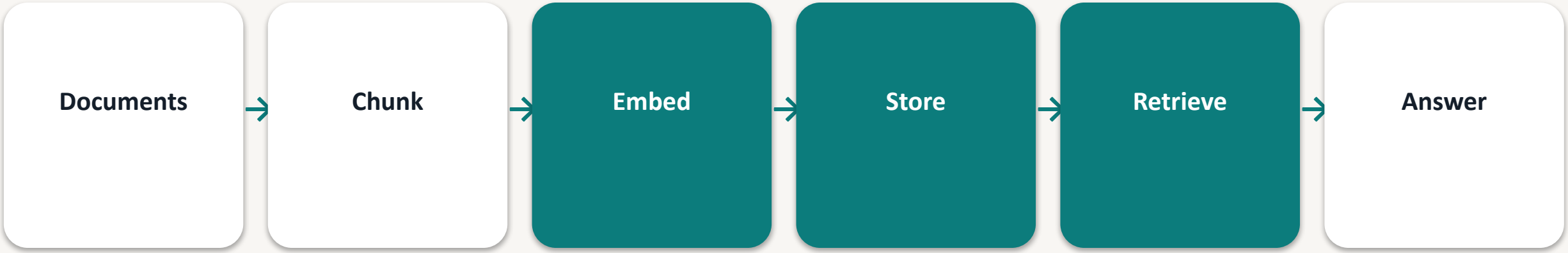
What we'll cover

From vectors → search → better answers

01	What embeddings are (and a tiny numeric example)
02	Similarity: how "near" becomes "relevant"
03	Indexing: embed chunks, store vectors + metadata
04	Retrieval: dense, hybrid, top-k, rerank — with examples
05	Failure modes, evaluation, and a classroom lab

Where we left off — and what's next

Chunking decides the units. Embeddings + retrieval decide which units you get.



Today's focus: turn chunks into vectors, store them, then find the best ones for a question.
Example corpus throughout: Employee Handbook (Remote Work, Benefits, Security).

What is an embedding?

A list of numbers that captures meaning — so similar text lands near similar text

In plain English

- Input: a chunk of text (or a query)
- Model: an embedding model (not the chat LLM)
- Output: a vector, e.g. 384 or 1536 floats
- Property: meaning-near ⇒ vector-near
- Used for: similarity search, clustering, rerank features

Mental model

"Equipment stipend is $500/year"
↓ embed
[0.12, -0.44, 0.91, … 0.03]

Same idea in different words
lands nearby in that space.

Example · Tiny 2D embedding space

x and y here are a teaching picture — not the real embedding format

Closer points = more related meaning. Search needs coordinates so a computer can rank “near” vs “far.”

On this slide (hypothetical)

Each sentence is a fake 2D point (x, y)
so we can draw “near” on a flat map.

In real RAG

Same idea, many more axes — e.g. 384 or 1536 numbers:
[0.12, −0.44, 0.91, …] (you don’t plot these by hand)

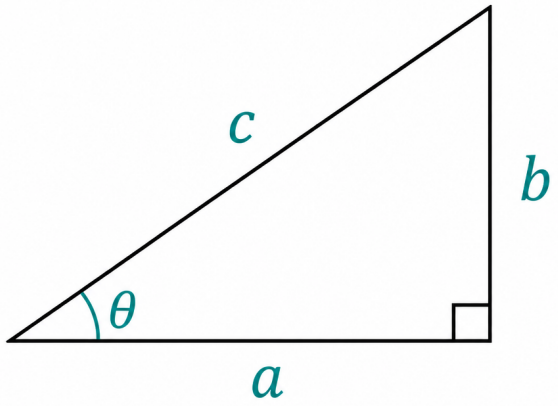
A Remote work: 3 days/week	Policy
B Manager approval by Friday	Policy
C Equipment stipend $500	Money
D Vision coverage after 90 days	Benefits
E MFA required for VPN	Security
Q "How much is the stipend?"	Query

Takeaway: Q should land nearest to C. We need coordinates for search — x,y is just the 2D cartoon of that.

Similarity: cosine (the usual default)

Score from -1 to 1 (often 0–1 after normalization). Higher = more alike.

Geometry reminder



$$\cos(\theta) = \frac{a}{c}$$

Closer angle θ → larger cos(θ)

In RAG (vectors)

cosine(q, d) =
(q · d) / (|q| |d|)

Same idea: angle between query vector and chunk vector.

- 1.0 nearly identical meaning
- 0.7–0.9 strong match
- < 0.4 usually off-topic

Why use it?

- Ignores vector length
- Focuses on direction = meaning
- Embeddings often normalized
- Then cosine ≈ dot product
- Vector DBs optimize for this

Example · Rank three chunks for one query

Toy 3D vectors (made-up numbers) — same process at 1536-D

Query Q: "How much is the equipment stipend?" → q = [0.90, 0.20, 0.10]

Chunk	Text (short)	Vector	Cosine ≈
C stipend	Equipment stipend $500/year	[0.88, 0.25, 0.08]	0.99
C remote	Remote work up to 3 days/week	[0.20, 0.85, 0.15]	0.41
C MFA	MFA required before VPN access	[0.10, 0.15, 0.90]	0.22

Retrieval returns C stipend first (top-1). Top-k=3 would still include weaker neighbors for the LLM to ignore.

Embedding models you'll meet

Different sizes, languages, and price — same job: text → vector

OpenAI / Azure text-embedding-3

API · Foundry or OpenAI · Small 1536 / Large 3072 dims

Common prod default in Azure AI Foundry

Cohere embed-v3

API · strong retrieval · 1024 dims

Good multilingual + search modes

all-MiniLM-L6-v2

Local · free · 384 dims

Labs, demos, offline apps

bge / e5 family

Local or hosted · 768+ dims

Strong open retrieval baselines

What does “dims” mean?

Dims = dimensions = how many numbers are in each embedding vector

Example: 1536 dims → each chunk/query becomes a list of 1536 numbers (a fingerprint of meaning).

384 dims

MiniLM-style

Smaller · faster · fine for labs

1536 dims

embedding-3-small

Common Azure / OpenAI default

3072 dims

embedding-3-large

Richer fingerprint · heavier

More dims can mean richer meaning — but also more storage, cost, and index size. Not always “bigger = must use.”

Azure AI Foundry · Small vs Large embeddings

Same job (text → vector). Different size, quality, and cost.

Text Embedding Small

Usually text-embedding-3-small

- Default dims: 1536
- Cheaper and faster enough for most RAG
- Great starting point for learning / prod
- Analogy: a clear phone photo

Text Embedding Large

Usually text-embedding-3-large

- Default dims: 3072
- Often better on hard retrieval questions
- Costs more; vectors use more storage
- Analogy: a higher-resolution photo

Rule: use the SAME model for indexing docs and embedding queries. Don’t mix Small docs + Large queries.

How to choose an embedding model

Start with a known retrieval model — then measure on your questions

Prototype / class lab	MiniLM or Foundry Small	Fast, cheap, good enough to learn
Most Azure RAG apps	text-embedding-3-small	Cost vs quality sweet spot in Foundry
Hard retrieval / high stakes	text-embedding-3-large	Try when Small misses too often
Multilingual handbook	Cohere / multilingual e5	Don't assume English-only models
Must stay on-prem	Local bge / e5	No API data leaving the building

Indexing: chunk → embed → store

Runs once (or on every doc update). Queries never re-embed the whole corpus.

1. Chunk

Split handbook into units (from Part 1)

2. Embed

Call embedding API / local model per chunk

3. Attach meta

source, section, page, parent_id, dates

4. Upsert

Write vector + meta into vector store

Rule: same embedding model at index time and query time. Mixing models breaks similarity.

Example · Indexing three handbook chunks

What actually gets written to the store

id	text	section	vector (abbrev)	embedded?
c1	Remote work up to 3 days/week	Remote	[0.21, 0.84, …]	Yes
c2	Equipment stipend is $500/year	Remote	[0.88, 0.24, …]	Yes
c3	MFA required before VPN access	Security	[0.11, 0.14, …]	Yes
p_remote	Full Section 3 body (parent)	Remote	—	No*

* Parent-child reminder: parents may be stored as plain text only; children carry the vectors used for search.

What is a vector store?

A database optimized to find nearest neighbors among many vectors

Stores

Vectors + payload (metadata + often the text)

Searches

ANN: approximate nearest neighbor (HNSW, IVF…)

Filters

"section = Security" before/after ANN

Examples

Pinecone, Weaviate, Qdrant, pgvector, Chroma

Example · Metadata filters save bad hits

Query: "VPN login rules" — without filters, Benefits chunks can sneak in

No filter (top-3)

- 1. MFA required before VPN ✓
- 2. Vision coverage after 90 days ✗
- 3. Device wipe on offboarding ~
- Problem: "access" words confuse Benefits

Filter: section = "Security"

- 1. MFA required before VPN ✓
- 2. VPN only on company devices ✓
- 3. Device wipe on offboarding ✓
- Same query — cleaner context for the LLM

Query time: retrieve → (optional) rerank → prompt

This path runs on every user question

1	**User asks**	"How much is the equipment stipend?"
2	**Embed query**	Same model as index → q vector
3	**ANN search**	Top-k nearest chunk vectors
4	**Optional rerank**	Cross-encoder rescoring
5	**Build prompt**	Stuff top passages + ask LLM
6	**Answer + cite**	Return text + source ids

Example · Dense retrieval end-to-end

One question, three scored neighbors, one answer

Q: "How much is the equipment stipend?"

#1 score 0.91

c2 Equipment stipend is $500/year

#2 score 0.62

c1 Remote work up to 3 days/week

#3 score 0.41

c4 Managers approve schedules by Friday

Prompt gets #1–#3. LLM answers: "$500 per year" and can cite c2.
If you only passed #2–#3 (bad top-k), the model may guess or say it doesn't know.

Top-k: how many chunks to fetch?

Too few misses context; too many wastes tokens and adds noise

k too small (e.g. 1–2)

- Miss second fact in multi-hop Qs
- Brittle if #1 is slightly wrong
- Good for ultra-precise factoids

k balanced (e.g. 4–8)

- Room for related neighbors
- Fits most chat context budgets
- Default starting point for labs

k too large (e.g. 20+)

- Noise dilutes the prompt
- Higher cost / latency
- Rerank becomes almost required

Example · Same query, different k

Q: "Stipend amount and who approves remote days?"

k = 1

- Returns only stipend chunk
- LLM knows $500
- Misses "manager approval by Friday"
- Half-answer — looks like a model bug

k = 4 (+ overlap/parent)

- Stipend + approval both present
- Full answer possible in one shot
- Still small enough for a clean prompt
- Fix was retrieval design, not a bigger LLM

Hybrid search: vectors + keywords

Dense finds paraphrase; sparse finds exact IDs, codes, names

Dense (embeddings)

- Great at synonyms & paraphrase
- "pay for laptop" ≈ "equipment stipend"
- Weak at rare tokens / SKUs
- Needs a good embedding model

Sparse (BM25 / keyword)

- Great at exact strings
- Policy-ID-442, error codes, names
- Weak when wording differs a lot
- Often free / already in your search stack

Example · Why hybrid wins on IDs

Q: "What does policy RW-12 say about stipends?"

Method	Top hit	Why
Dense only	Generic stipend paragraph	Misses rare token RW-12
BM25 only	Doc titled RW-12	May miss paraphrased body
Hybrid (fuse)	RW-12 section with $500 text	ID match + meaning match

Common fuse: Reciprocal Rank Fusion (RRF) — merge ranked lists without fragile score calibration.

Reranking: a second, smarter sort

Retrieve broad (cheap), then rescore top candidates (precise)

Retrieve

Fetch k=20 with ANN / hybrid

Rerank

Cross-encoder scores (query, chunk) pairs

Keep

Pass top 3–5 to the LLM

Why it helps: bi-encoders embed query & doc separately (fast). Cross-encoders read both together (accurate, slower).
Use when first-stage recall is good but precision@5 is messy.

Example · Rerank fixes a near-miss

Q: "Is security training required before VPN?"

After ANN (k=5 order)

- 1. VPN only on company devices
- 2. MFA required before VPN
- 3. Security training before access ← true answer
- 4. Remote work 3 days/week
- 5. Device wipe policy

After cross-encoder rerank

- 1. Security training before access ✓
- 2. MFA required before VPN
- 3. VPN only on company devices
- …
- LLM now sees the decisive sentence first

Diversity: MMR (Maximal Marginal Relevance)

Avoid sending five near-duplicate chunks about the same sentence

Without MMR

- Top hits all paraphrase "$500 stipend"
- Approval rule never appears
- Wastes context window
- Multi-hop answers fail quietly

With MMR

- Keeps best stipend chunk
- Prefers a different but still relevant chunk
- Approval + stipend both in prompt
- Trade a bit of pure similarity for coverage

Common retrieval failures (with fixes)

These look like "the LLM is dumb" — usually it never saw the right text

Wrong embedding model mix

Re-index with one model end-to-end

Chunk too big / too small

Revisit Part 1 strategies

No metadata filters

Filter by product, locale, section

k too low for multi-hop

Raise k or use parent-child

Exact ID queries

Add BM25 / hybrid

Good recall, bad order

Add a reranker

Evaluate retrieval before blaming the LLM

Build 20–50 labeled questions; score the search step alone

Recall@k	Was the needed chunk in the top-k?	Must-have for RAG
MRR	How high was the first relevant hit?	Order quality
nDCG	Graded relevance across the list	When multiple goods exist
Answer accuracy	Did the final LLM answer match gold?	End-to-end check

Example · Scoring three strategies on 10 questions

Same handbook corpus — only retrieval setup changes

Setup	Recall@5	MRR	What happened
Dense, k=5	0.70	0.55	Missed RW-12 ID questions
Hybrid, k=5	0.85	0.68	IDs fixed; still some order noise
Hybrid + rerank	0.90	0.82	Best precision for the prompt

Teaching takeaway: measure retrieval first. Don't swap LLMs until Recall@k is healthy.

Retrieval toolkit at a glance

Layer techniques as quality needs grow

Technique	Helps with	Cost	Start here?
Dense ANN	Paraphrase / meaning	Low–Med	Yes
Metadata filters	Wrong-section noise	Low	Yes
Hybrid + RRF	IDs, codes, names	Med	Often
Reranker	Precision of top-5	Med–High	When noisy
MMR	Duplicate neighbors	Low	Multi-hop
Parent expand	Thin children	Low	Long manuals

How to choose a retrieval setup

Start simple; upgrade when evals fail a specific way

Shipping a class demo this week?	→ Dense + k=5 + citations
Users type product IDs / error codes?	→ Add hybrid (BM25 + dense)
Top-k has the answer but buried?	→ Add reranker
Answers need a whole section?	→ Parent-child expand after hit
Many near-duplicate chunks?	→ MMR or better chunking
Wrong department docs appear?	→ Metadata filters first

Applications · retrieval patterns in the wild

Same embedding idea — different retrieval recipes

Support bot

Hybrid + filters by product

Cite help-center URL

HR handbook Q&A

Dense + parent expand

Policy section in prompt

Code assistant

Code embeddings + path filters

Don't mix random files

Legal search

Hybrid + rerank

Clause IDs must hit

Research papers

Dense + section metadata

Methods vs results

Internal wiki

Hybrid + recency boost

Prefer newer pages

Classroom lab (40–50 min)

Same corpus as chunking lab — now score retrieval

1. Reuse the handbook chunks from Part 1 (or re-chunk once)
2. Embed with one model; store vectors + metadata
3. Write 12 questions: 4 easy, 4 paraphrase, 2 ID, 2 multi-hop
4. Run dense-only vs hybrid; measure Recall@5
5. Optional: add rerank; compare MRR and answer quality
6. Discuss: which failure was retrieval vs chunking vs LLM?

Key takeaways

What students should leave remembering

Embeddings = meaning as numbers

Same model at index + query

Retrieval is a product decision

k, filters, hybrid, rerank

Measure search separately

Recall@k before swapping LLMs

Examples beat slogans

Always debug with a real question

NEXT UP

Augmenting prompts, citations, and RAG evaluation

Questions? Run the lab — then we'll grade your retrievers, not just your prompts.